

UNIT-V

LEETCODE EXERCISE

1.Product Sales Analysis I

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

- (sale_id, year) is the primary key of this table.
- product_id is a foreign key to Product table.
- Note that the price is per unit.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

- product_id is the primary key of this table .

Q:Write an SQL query that reports the product_name, year, and price for each sale_id in the Sales table .

Return the resulting table in any order.

</> Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select p.product_name, s.year, s.price
3 from Sales s
4 left join Product p
5 on s.product_id = p.product_id
```

✓ Testcase > Test Result

Accepted Runtime: 100 ms

✓ Case 1

Input

Sales =

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product =

product_id	product_name
100	Nokia
200	Apple
300	Samsung

Output

product_name	year	price
Nokia	2008	5000
Nokia	2009	5000
Apple	2011	9000

Explanation:

From sale_id = 1, we can conclude that Nokia was sold for 5000 in the year 2008.

From sale_id = 2, we can conclude that Nokia was sold for 5000 in the year 2009.

From sale_id = 7, we can conclude that Apple was sold for 9000 in the year 2011.

2.Product Sales Analysis II

Table: Sales

Column Name	Type
Sale_id	int
Product_id	int
year	int
quantity	int
price	int

- sale_id is the primary key of this table.
- product_id is a foreign key to Product table.
- Note that the price is per unit.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key of this table.

Q:Write an SQL query that reports the total quantity sold for every product id.

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select product_id, sum(quantity) as total_quantity
3 from Sales
4 group by product_id;
```

 Testcase  Test Result

Output

product_id	total_quantity
100	22
200	15

3.Sales Analysis III

Table: Product

Column Name	Type
product_id	int
product_name	int
unit_price	int

product_id is the primary key of this table.

Table: Sales

Column Name	Type
seller_id	int
product_id	int
buyer_id	int
sale_date	int
quantity	int
price	int

- This table has no primary key, it can have repeated rows.
- product_id is a foreign key to Product table.

Q:write an SQL query that reports the products that were only sold in spring 2019.

That is, between 2019-01-01 and 2019-03-31 inclusive.

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select p.product_id, p.product_name
3 from Product p
4 left join Sales s
5 on p.product_id = s.product_id
6 group by p.product_id
7 having sum(s.sale_date between '2019-01-01' and '2019-03-31') = count(s.sale_date)
```

✓ Testcase | > Test Result

Accepted Runtime: 105 ms

✓ Case 1

Input

Product =

product_id	product_name	unit_price
1	S8	1000
2	G4	800
3	iPhone	1400

Sales =

seller_id	product_id	buyer_id	sale_date	quantity	price
1	1	1	2019-01-21	2	2000
1	2	2	2019-02-17	1	800
2	2	3	2019-06-02	1	800
3	3	4	2019-05-13	2	2800

Output

product_id	product_name
1	S8

Explanation:

The product with id 1 was only sold in the spring of 2019.

The product with id 2 was sold in the spring of 2019 but was also sold after the spring of 2019.

The product with id 3 was sold after spring 2019.

We return only product 1 as it is the product that was only sold in the spring of 2019.

4.Product Price At A Given Date

Table: Products

Column Name	Type
product_id	int
new_price	int
change_date	date

- (product_id, change_date) is the primary key (combination of columns with unique values) of this table.
- Each row of this table indicates that the price of some product was changed to a new price at some date.
- Initially, all products have price 10.

Q: Write a solution to find the prices of all products on the date 2019-08-16.

</> Code

MySQL   Auto

```
1  # Write your MySQL query statement below
2  WITH LatestPrice AS (
3      SELECT
4          product_id,
5          new_price,
6          ROW_NUMBER() OVER (
7              PARTITION BY product_id
8              ORDER BY change_date DESC
9          ) AS rn
10     FROM Products
11     WHERE change_date <= '2019-08-16'
12 )
13
14 SELECT
15     p.product_id,
16     COALESCE(lp.new_price, 10) AS price
17 FROM
18     (SELECT DISTINCT product_id FROM Products) p
19 LEFT JOIN LatestPrice lp
20     ON p.product_id = lp.product_id
21     AND lp.rn = 1;
```

✓ Testcase > Test Result

Accepted Runtime: 64 ms

✓ Case 1

Input

Products =

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17
3	20	2019-08-18

Output

product_id	price
1	35
2	50
3	10

5.Immediate Food Delivery II

Table: Delivery

Column Name	Type
delivery_id	int
customer_id	int
order_date	date
customer_pref_delivery_date	date

delivery_id is the column of unique values of this table.

The table holds information about food delivery to customers that make orders at some date and specify a preferred delivery date (on the same order date or after it).

If the customers preferred delivery date is the same as the order date, then the order is

called immediate; otherwise, it is called scheduled.

The first order of a customer is the order with the earliest order date that the customer made. It is guaranteed that a customer has precisely one first order.

Write a solution to find the percentage of immediate orders in the first orders of all customers, rounded to 2 decimal places.

 Code

MySQL   Auto

```
1  # Write your MySQL query statement below
2  WITH FirstOrders AS (
3      SELECT *,
4          ROW_NUMBER() OVER (
5              PARTITION BY customer_id
6              ORDER BY order_date
7          ) AS rn
8      FROM Delivery
9  )
10
11 SELECT
12     ROUND(
13         100.0 *
14         SUM(CASE
15             WHEN order_date = customer_pref_delivery_date
16             THEN 1
17             ELSE 0
18         END)
19         / COUNT(*),
20         2
21     ) AS immediate_percentage
22 FROM FirstOrders
23 WHERE rn = 1;
```

✓ Testcase > Test Result

Accepted Runtime: 88 ms

✓ Case 1

Input

Delivery =

delivery_id	customer_id	order_date	customer_pref_delivery_date
1	1	2019-08-01	2019-08-02
2	2	2019-08-02	2019-08-02
3	1	2019-08-11	2019-08-12
4	3	2019-08-24	2019-08-24
5	3	2019-08-21	2019-08-22
6	2	2019-08-11	2019-08-13
7	4	2019-08-09	2019-08-09

Output

immediate_percentage
50

Explanation:

The customer id 1 has a first order with delivery id 1 and it is scheduled.

The customer id 2 has a first order with delivery id 2 and it is immediate.

The customer id 3 has a first order with delivery id 5 and it is scheduled.

The customer id 4 has a first order with delivery id 7 and it is immediate.

Hence, half the customers have immediate first orders.

6.Monthly Transactions I

Table: Transactions

Column Name	Type
id	int
country	varchar
state	enum
amount	int
trans_date	date

id is the primary key of this table.

The table has information about incoming transactions.

The state column is an enum of type ["approved", "declined"].

Q:Write an SQL query to find for each month and country, the number of transactions and their total amount, the number of approved transactions and their total amount.

Return the result table in any order.

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     DATE_FORMAT(trans_date, '%Y-%m') AS month,
4     country,
5     COUNT(*) AS trans_count,
6     SUM(CASE
7         WHEN state = 'approved' THEN 1
8         ELSE 0
9     END) AS approved_count,
10    SUM(amount) AS trans_total_amount,
11    SUM(CASE
12        WHEN state = 'approved' THEN amount
13        ELSE 0
14    END) AS approved_total_amount
15 FROM Transactions
16 GROUP BY DATE_FORMAT(trans_date, '%Y-%m'), country;
```

☒ Testcase  Test Result

Accepted Runtime: 70 ms

☒ Case 1

Input

Transactions =

id	country	state	amount	trans_date
121	US	approved	1000	2018-12-18
122	US	declined	2000	2018-12-19
123	US	approved	2000	2019-01-01
124	DE	approved	2000	2019-01-07

Output

month	country	trans_count	approved_count	trans_total_amount	approved_total_amount
2018-12	US	2	1	3000	1000
2019-01	US	1	1	2000	2000
2019-01	DE	1	1	2000	2000

7.Last Person to Fit in the Bus

Table: Queue

Name	Type
person_id	int
person_name	varchar
weight	int
turn	int

person_id column contains unique values. This table has the information about all people waiting for a bus.

The person_id and turn columns will contain all numbers from 1 to n, where n is the number of rows in the table.

turn determines the order of which the people will board the bus, where turn=1 denotes the first person to board and turn=n denotes the last person to board.

weight is the weight of the person in kilograms.

Q: There is a queue of people waiting to board a bus.

However, the bus has a weight limit of 1000 kilograms, so there may be some people who cannot board.

Write a solution to find the person_name of the last person that can fit on the bus without exceeding the weight limit.

The test cases are generated such that the first person does not exceed the weight limit. Note that only one person can board the bus at any given turn.

</> Code

MySQL   Auto

```
1  # Write your MySQL query statement below
2  WITH RunningTotal AS (
3      SELECT
4          person_name,
5          turn,
6          SUM(weight) OVER (ORDER BY turn) AS total_weight
7      FROM Queue
8  )
9
10 SELECT person_name
11 FROM RunningTotal
12 WHERE total_weight <= 1000
13 ORDER BY turn DESC
14 LIMIT 1;
```

☒ Testcase  Test Result

Accepted Runtime: 76 ms

☒ Case 1

Input

Queue =

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

Output

person_name
John Cena

8. Restaurant Growth

Table: Customer

Column Name	Type
customer_id	int
name	varchar
visited_on	date
amount	int

In SQL, (customer_id, visited_on) is the primary key for this table.

This table contains data about customer transactions in a restaurant.

visited_on is the date on which the customer with ID (customer_id) has visited the restaurant.

amount is the total paid by a customer.

Q: You are the restaurant owner and you want to analyze a possible expansion (there will be at least one customer every day).

Compute the moving average of how much the customer paid in a seven days window (i.e., current day + 6 days before).

average_amount should be rounded to two decimal places.

Return the result table ordered by visited_on in ascending order.

</> Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 WITH daily AS (
3     SELECT
4         visited_on,
5         SUM(amount) AS daily_total
6     FROM Customer
7     GROUP BY visited_on
8 )
9
10 SELECT
11     d1.visited_on,
12     SUM(d2.daily_total) AS amount,
13     ROUND(SUM(d2.daily_total) / 7, 2) AS average_amount
14 FROM daily d1
15 JOIN daily d2
16     ON d2.visited_on BETWEEN DATE_SUB(d1.visited_on, INTERVAL 6 DAY)
17     AND d1.visited_on
18 GROUP BY d1.visited_on
19 HAVING COUNT(d2.visited_on) = 7
20 ORDER BY d1.visited_on;
```

☒ Testcase  Test Result

Accepted Runtime: 66 ms

☒ Case 1

Input

Customer =

customer_id	name	visited_on	amount
1	Jhon	2019-01-01	100
2	Daniel	2019-01-02	110
3	Jade	2019-01-03	120
4	Khaled	2019-01-04	130
5	Winston	2019-01-05	110
6	Elvis	2019-01-06	140

 View more

Output

visited_on	amount	average_amount
2019-01-07	860	122.86
2019-01-08	840	120
2019-01-09	840	120
2019-01-10	1000	142.86

Explanation:

1st moving average from 2019-01-01 to 2019-01-07 has an average_amount of $(100 + 110 + 120 + 130 + 110 + 140 + 150)/7 = 122.86$

2nd moving average from 2019-01-02 to 2019-01-08 has an average_amount of $(110 + 120 + 130 + 110 + 140 + 150 + 80)/7 = 120$

3rd moving average from 2019-01-03 to 2019-01-09 has an average_amount of $(120 + 130 + 110 + 140 + 150 + 80 + 110)/7 = 120$

4th moving average from 2019-01-04 to 2019-01-10 has an average_amount of $(130 + 110 + 140 + 150 + 80 + 110 + 130 + 150)/7 = 142.86$

9.Movie Rating

Table: Movies

Column Name	Type
movie_id	int
movie_id	varchar

movie_id is the primary key (column with unique values) for this table.

title is the name of the movie.

Each movie has a unique title.

Table: Users

Column Name	Type
user_id	int
name	varchar

user_id is the primary key (column with unique values) for this table.

The column 'name' has unique values

Write a solution to:

Find the name of the user who has rated the greatest number of movies. In case of a tie, return the lexicographically smaller user name.

Find the movie name with the highest average rating in February 2020. In case of a tie, return the lexicographically smaller movie name.

 Code

MySQL   Auto

```
1  # Write your MySQL query statement below
2  (
3      -- Part 1: User who rated the most movies
4      SELECT u.name AS results
5      FROM MovieRating mr
6      JOIN Users u
7      |   ON mr.user_id = u.user_id
8      GROUP BY mr.user_id
9      ORDER BY COUNT(*) DESC, u.name ASC
10     LIMIT 1
11 )
12
13 UNION ALL
14
15 (
16     -- Part 2: Movie with highest average rating in Feb 2020
17     SELECT m.title AS results
18     FROM MovieRating mr
19     JOIN Movies m
20     |   ON mr.movie_id = m.movie_id
21     WHERE DATE_FORMAT(mr.created_at, '%Y-%m') = '2020-02'
22     GROUP BY mr.movie_id
23     ORDER BY AVG(mr.rating) DESC, m.title ASC
24     LIMIT 1
25 );
```

✓ Testcase | > Test Result

Accepted Runtime: 152 ms

✓ Case 1

Input

Movies =

movie_id	title
1	Avengers
2	Frozen 2
3	Joker

Users =

user_id	name
1	Daniel
2	Monica
3	Maria
4	James

MovieRating =

movie_id	user_id	rating	created_at
1	1	3	2020-01-12
1	2	4	2020-02-11
1	3	2	2020-02-12
1	4	1	2020-01-01
2	1	5	2020-02-17
2	2	2	2020-02-01

⌵ View more

Output

results
Daniel
Frozen 2

Explanation:

Daniel and Monica have rated 3 movies ("Avengers", "Frozen 2" and "Joker") but Daniel is smaller lexicographically.

Frozen 2 and Joker have a rating average of 3.5 in February but Frozen 2 is smaller lexicographically.